

Experimental Report on a Vision-Based Maze Drawing System Using a 5-DOF Robotic Arm

Project: Maze Robotic Arm Drawing System

June 15, 2026

Abstract

This experiment implements a complete robotic system that converts a camera image of a hand-drawn paper maze into an executable drawing trajectory for a 5-DOF robotic arm. The input is a white-paper maze drawn with black lines, where the start point is marked in red and the goal point is marked in blue. The system first reconstructs the paper maze through perspective correction and image binarization, then performs A* path planning on an occupancy grid, and finally maps the 2-D path into physical paper coordinates and joint-space commands. The real-arm execution is handled through a TCP client-server interface and a serial connection to the arm controller. The project demonstrates an integrated robotics workflow that combines computer vision, graph search, forward/inverse kinematics, simulation, and real hardware control.

Contents

1	Experiment Overview	4
2	Hardware and Software Platform	5
2.1	Hardware Platform	5
2.2	Software Platform	7
3	Overall System Design	7
4	Project Structure and Module Relationship	7

5	Maze Image Reconstruction	8
5.1	Paper Detection and Perspective Correction	8
5.2	Illumination Enhancement and Binarization	11
6	Path Planning	13
6.1	Mathematical Formulation	13
6.2	Start and Goal Detection	14
6.3	Occupancy Grid Construction	14
6.4	A* Search	16
7	Kinematics and Trajectory Generation	18
7.1	Pixel-to-Paper Coordinate Mapping	18
7.2	Arc-Length Resampling	19
7.3	Forward Kinematics	20
7.4	Inverse Kinematics	20
8	Isaac Sim Verification	22
9	Real Robotic Arm Control	23
9.1	Communication Architecture	23
9.2	TCP Client	24
9.3	Serial Command Generation	24
9.4	Trajectory Execution Strategy	25
9.5	Real-Arm Trajectory Preprocessing	25
10	Experimental Procedure	26
10.1	Maze Planning	26
10.2	Simulation	26
10.3	Real-Arm Dry Run	27
10.4	Real-Arm Execution	27
11	Results and Analysis	27
11.1	Image Processing Result	27

11.2 Path Planning Result	29
11.3 Kinematic Result	30
11.4 Real Hardware Result	30
12 Issues and Future Work	30
13 Conclusion	31
A Key Files	31

1 Experiment Overview

The goal of this project is to let a robotic arm draw the solution path of a maze from an ordinary camera photo. Since the photo may contain perspective distortion, uneven illumination, shadows, and background clutter, the system first reconstructs a clean top-down maze image. It then plans a collision-free path from the red start marker to the blue goal marker. The resulting 2-D path is converted into physical coordinates on the paper and then into five joint commands through inverse kinematics.

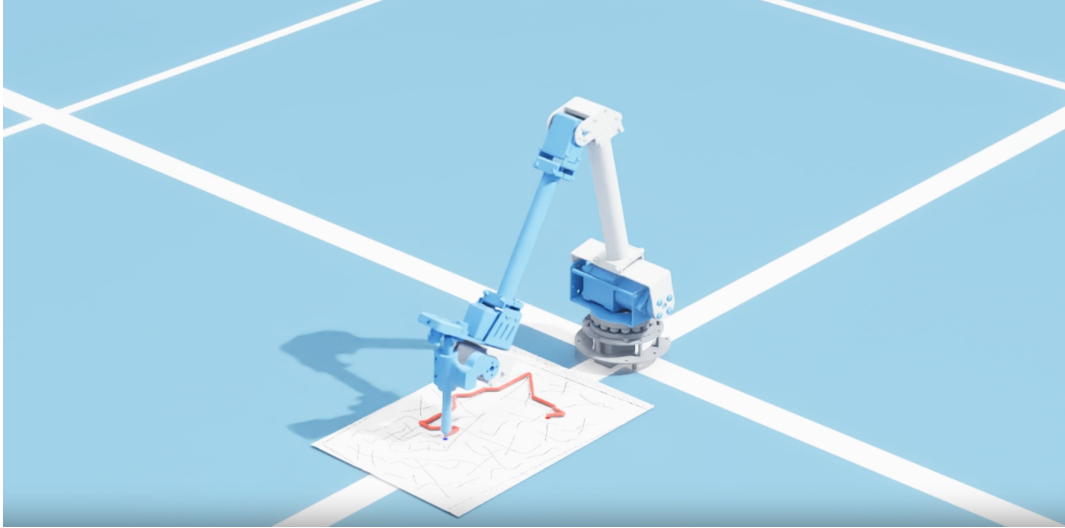


Figure 1: Overall project visualization

The system is divided into four major layers, as shown in Table 1.

Table 1: Main system modules

Module	Function	Main files
Vision recon- struction	Recover a top-down binary maze image from a tilted photo	<code>maze_planner/maze_scanner.py</code>
Path planning	Detect start/goal markers, build an occupancy grid, and run A* search	<code>maze_planner/maze_planner.py</code>
Simulation	Import the 5-DOF robot into Isaac Sim and verify drawing behavior	<code>arm_sim/draw_maze.py</code>
Real execution	Convert the path to joint commands and send them to the real arm	<code>arm_real_client/</code> , <code>arm_real_server/</code>

2 Hardware and Software Platform

2.1 Hardware Platform

The experiment uses a 5-DOF desktop robotic arm. The original gripper is replaced by a pen holder so that the robot end-effector can draw on a sheet of paper. The arm communicates with a Windows lower-level host through USB serial and requires a separate 12 V power supply.

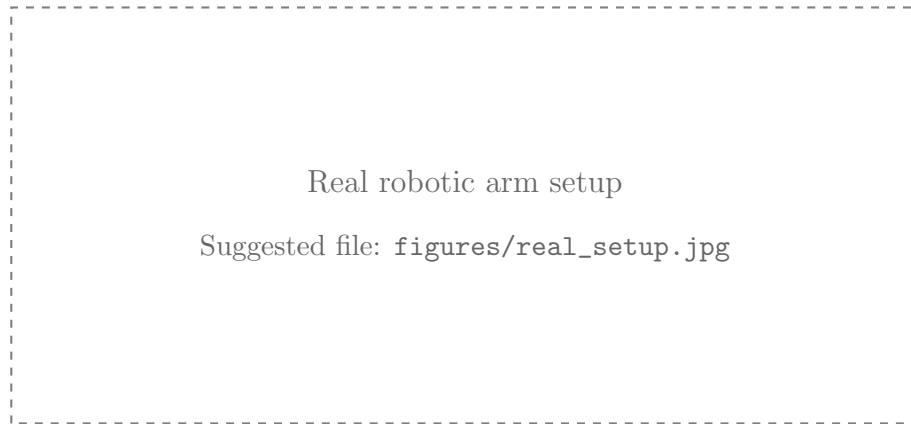


Figure 2: Real robotic arm setup, to be added

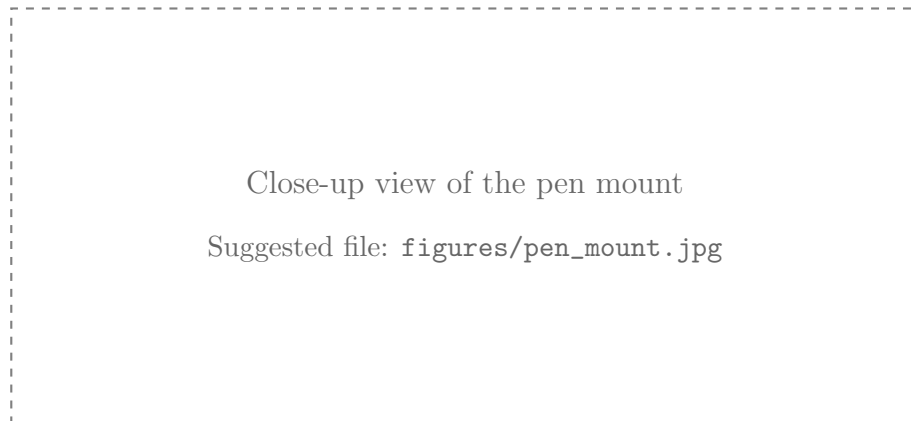


Figure 3: Close-up view of the pen mount, to be added

The URDF model of the robot is stored in `urdf/five_dof_arm.urdf`, and the corresponding STL meshes are stored in `urdf/meshes/`. Figure 4 and Figure 5 show the simulated zero pose and reachable workspace.

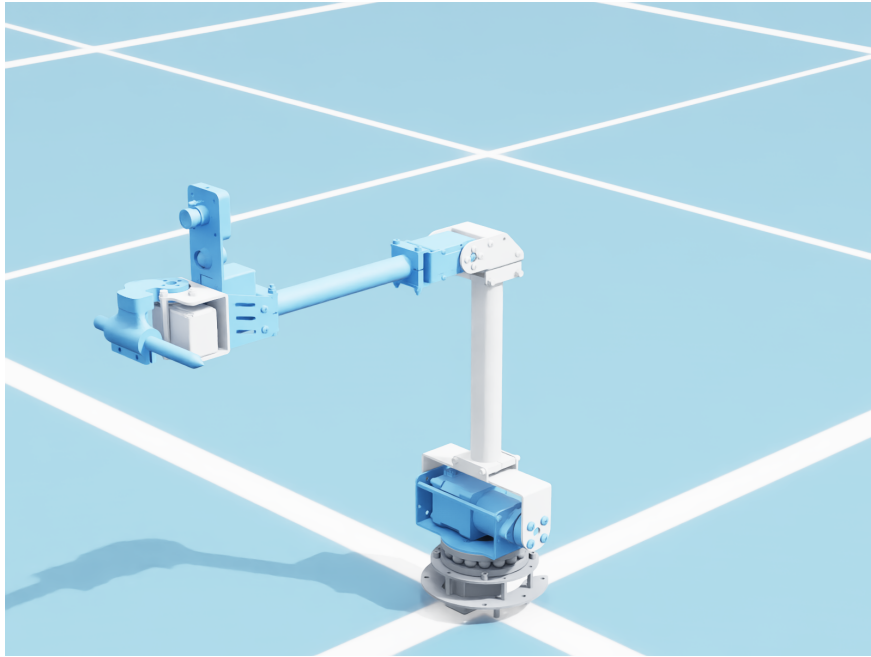


Figure 4: Zero pose of the robotic arm

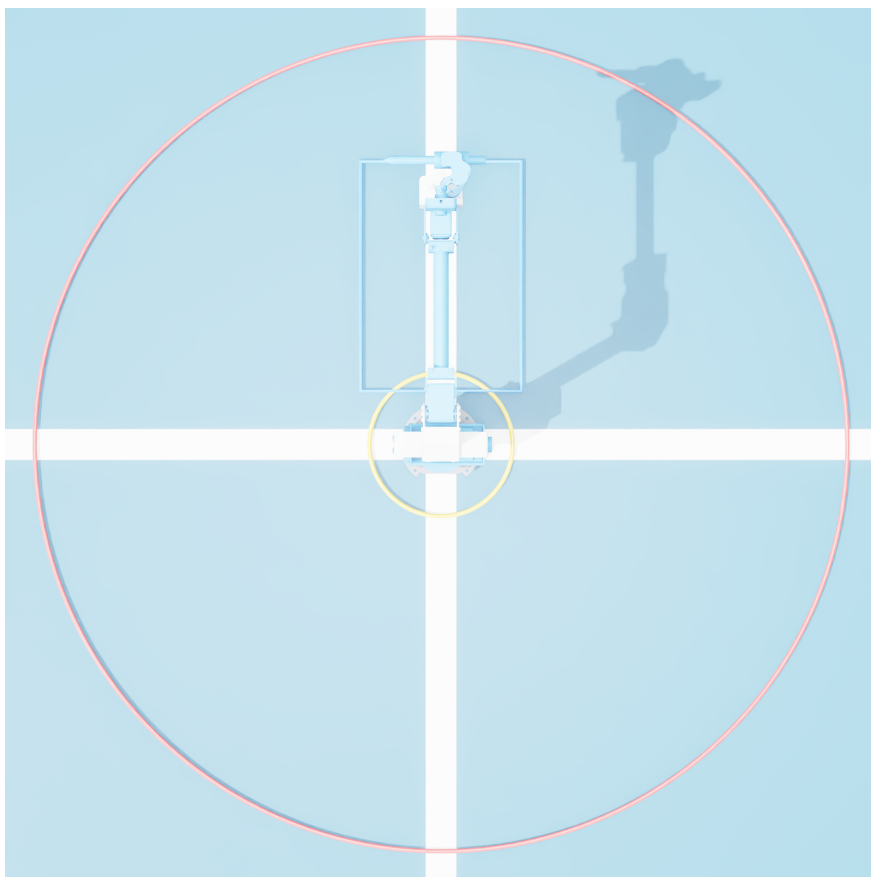


Figure 5: Reachable workspace of the robotic arm

2.2 Software Platform

The project is implemented mainly in Python. OpenCV is used for paper detection, perspective transformation, binarization, and colored marker detection. NumPy and SciPy are used for coordinate mapping, trajectory resampling, and inverse-kinematics optimization. Isaac Sim is used for simulation and video rendering. The real-arm control link is implemented with Python sockets and pyserial.

3 Overall System Design

The end-to-end workflow is shown in Figure 6.

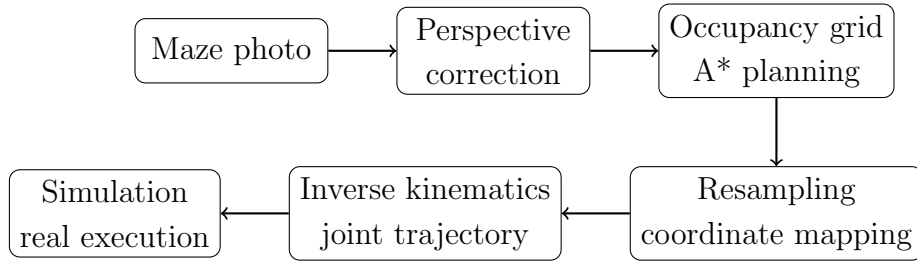


Figure 6: Overall system pipeline

The vision and planning modules generate a pixel-space path. Both the simulation and real-arm client reuse the same pixel-to-world mapping and the same kinematics implementation in `arm_kinematics.py`. This keeps the simulated and real trajectories consistent.

4 Project Structure and Module Relationship

The repository follows the robotics pipeline of perception, planning, kinematics, simulation, and real hardware control.

Listing 1: Main project structure

```
1 Maze/
2   maze_planner/ # Maze reconstruction and path planning
3     maze_scanner.py # Paper detection, perspective correction, binarization
4     maze_planner.py # Marker detection, occupancy grid, A* search
5     samples/ # Test maze images
6     outputs/ # Intermediate images and trajectory output
7
8   arm_sim/ # Isaac Sim simulation and kinematic validation
9     arm_kinematics.py # 5-DOF forward and inverse kinematics
10    draw_maze.py # Simulated maze drawing
```

```

11  draw_circle.py # FK/IK validation by drawing a circle
12  video/ # Rendered simulation videos
13
14  arm_real_client/ # Upper-level client
15  draw_maze_real.py # Image -> path -> IK -> TCP trajectory command
16  robot_client.py # TCP JSON client wrapper
17  estop.py # Software stop command
18
19  arm_real_server/ # Lower-level server
20  robot_server.py # TCP server, limits, point execution, feedback
21  robot_server_openloop.py # Open-loop timed execution version
22
23  urdf/ # Robot model
24  five_dof_arm.urdf
25  meshes/
26
27  assets/ # Demonstration images and simulation captures

```

The module relationship is shown in Figure 7. The planner produces a pixel path; the kinematics module converts it into joint angles; the simulation and real client consume the same joint trajectory.

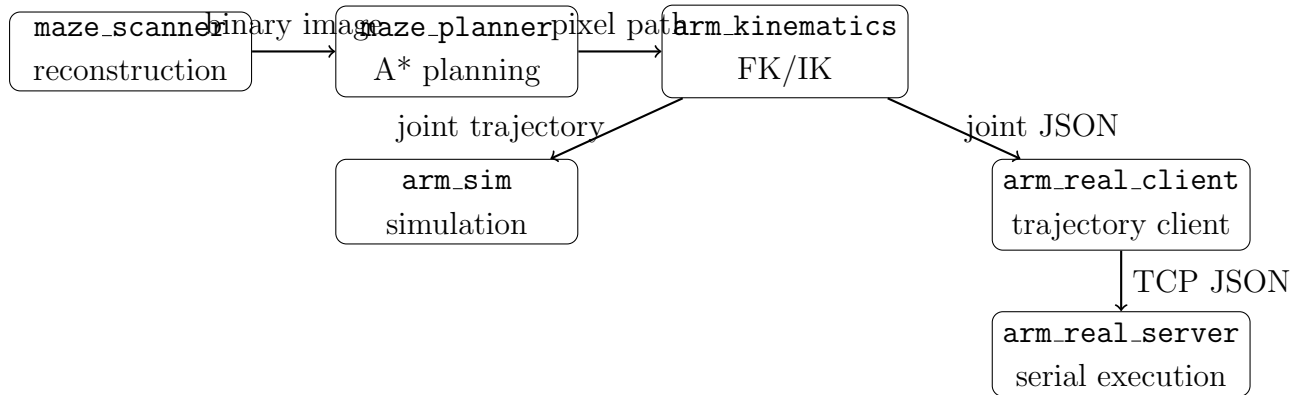


Figure 7: Module relationship

5 Maze Image Reconstruction

5.1 Paper Detection and Perspective Correction

The input photo may be tilted and affected by perspective distortion. The scanner first detects the paper region using brightness and saturation cues; if this fails, it falls back to edge-based detection. The four paper corners are ordered as top-left, top-right, bottom-right, and bottom-left, then a perspective transform is applied.

Listing 2: Four-point perspective transformation

```
1 def four_point_transform(img, rect):
2     tl, tr, br, bl = rect
3     width_top = np.linalg.norm(tr - tl)
4     width_bottom = np.linalg.norm(br - bl)
5     height_left = np.linalg.norm(bl - tl)
6     height_right = np.linalg.norm(br - tr)
7     max_w = int(round(max(width_top, width_bottom)))
8     max_h = int(round(max(height_left, height_right)))
9
10    dst = np.array([[0, 0],
11                    [max_w - 1, 0],
12                    [max_w - 1, max_h - 1],
13                    [0, max_h - 1]], dtype="float32")
14    M = cv2.getPerspectiveTransform(rect, dst)
15    return cv2.warpPerspective(img, M, (max_w, max_h))
```

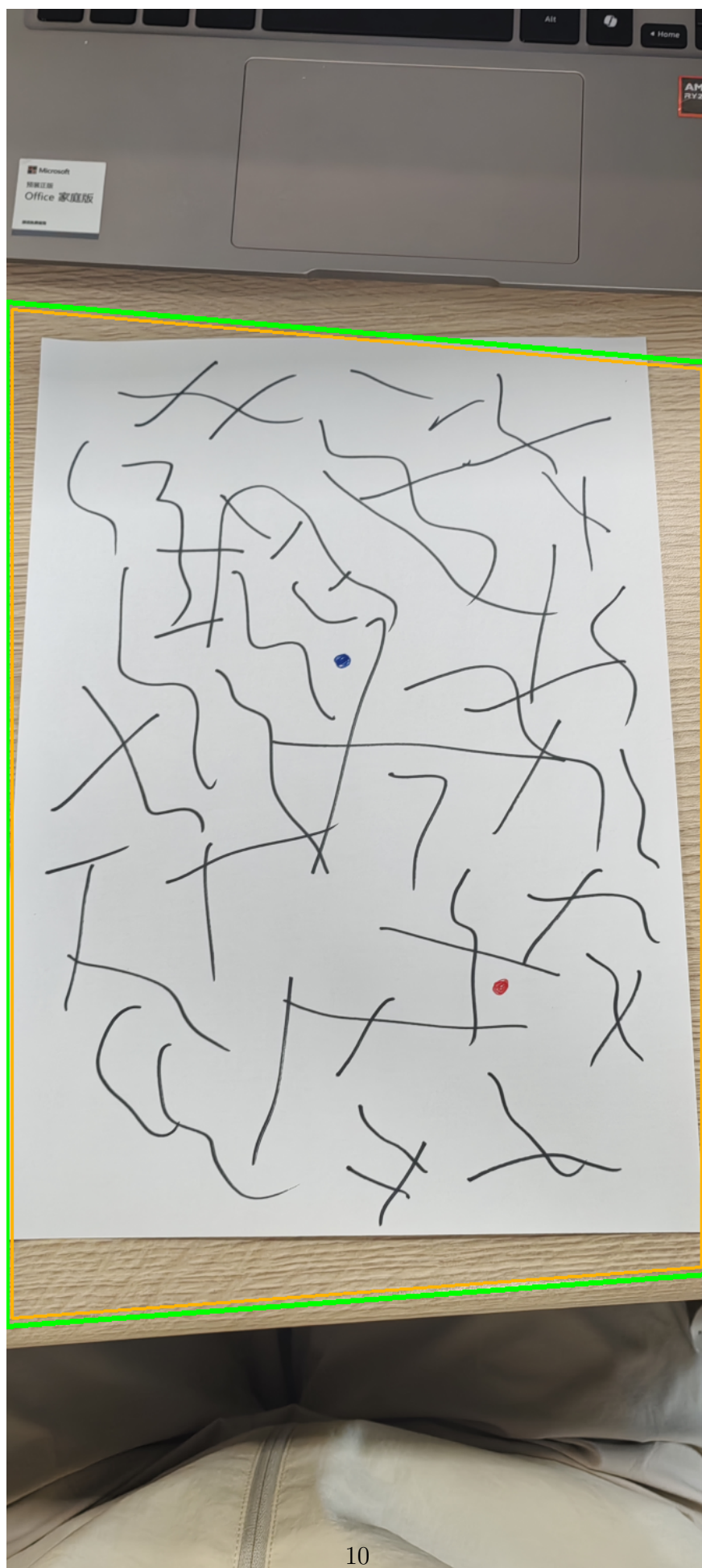


Figure 8: Detected paper contour

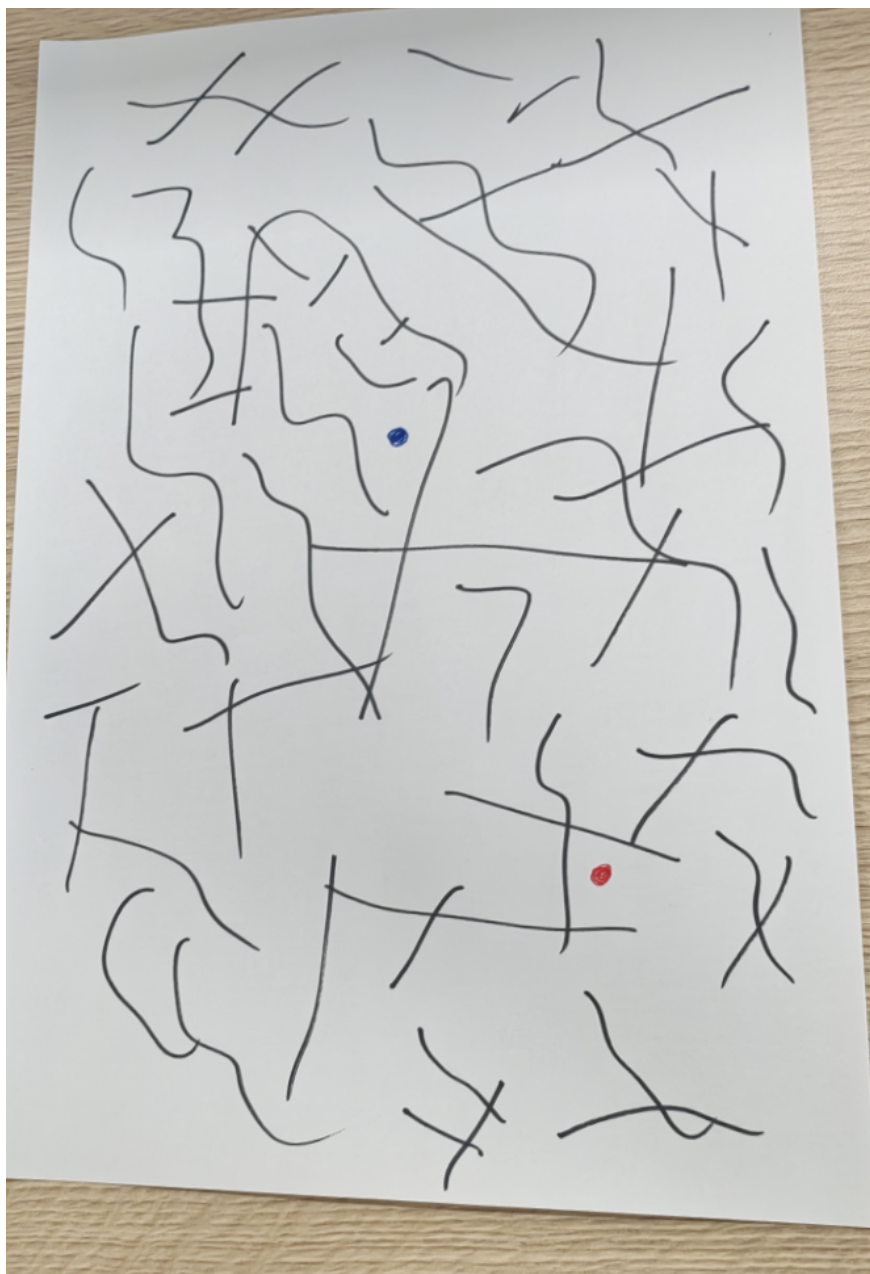


Figure 9: Perspective-corrected maze image

5.2 Illumination Enhancement and Binarization

Uneven lighting is handled by estimating a smooth background with a large Gaussian blur and normalizing the original grayscale image by this background. CLAHE is then applied to enhance local contrast, followed by adaptive thresholding.

Listing 3: Illumination enhancement

```

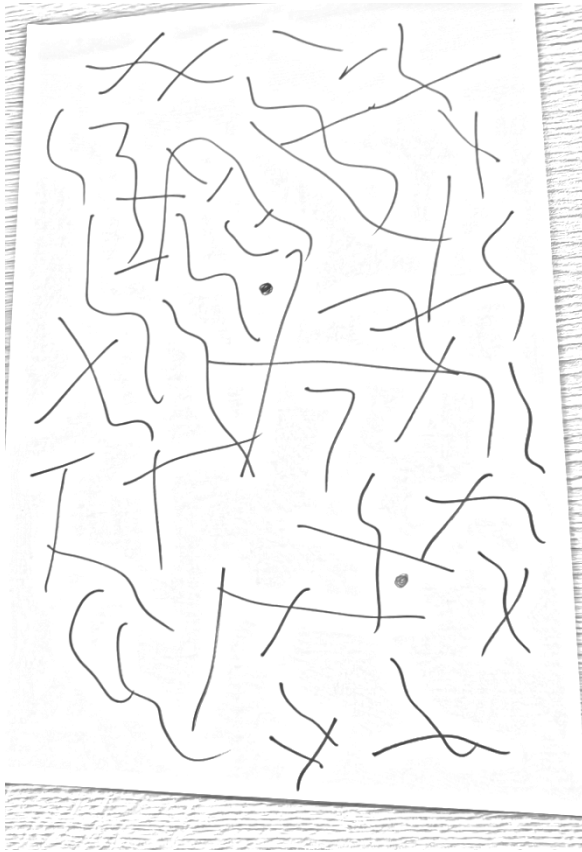
1 def enhance(img):
2     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) if img.ndim == 3 else img
3     k = max(31, (max(gray.shape) // 20) | 1)

```

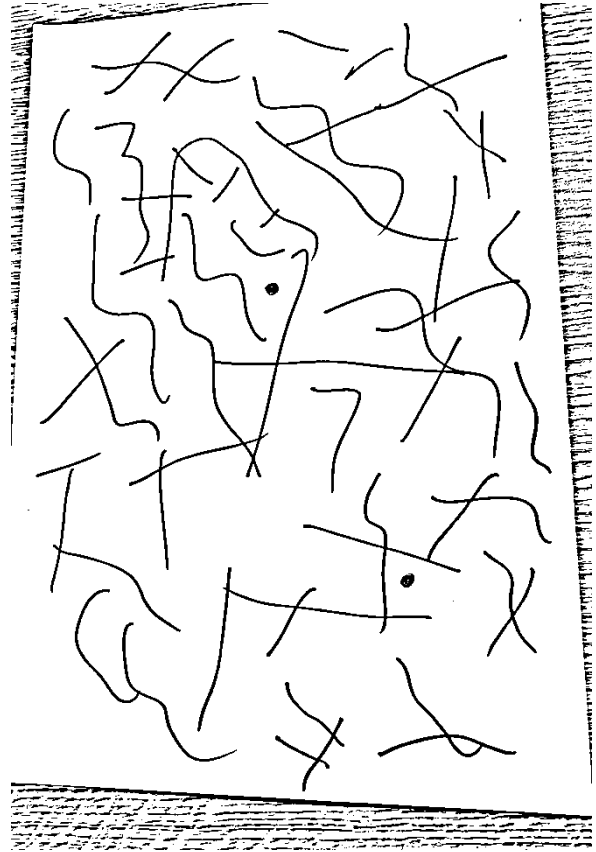
```

4 background = cv2.GaussianBlur(gray, (k, k), 0)
5 background = np.where(background == 0, 1, background)
6 norm = gray.astype(np.float32) / background.astype(np.float32)
7 norm = np.clip(norm, 0, 1)
8 norm = (norm * 255).astype(np.uint8)
9 clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
10 return clahe.apply(norm)

```



(a) Enhanced image



(b) Binary image

Figure 10: Enhancement and binarization results

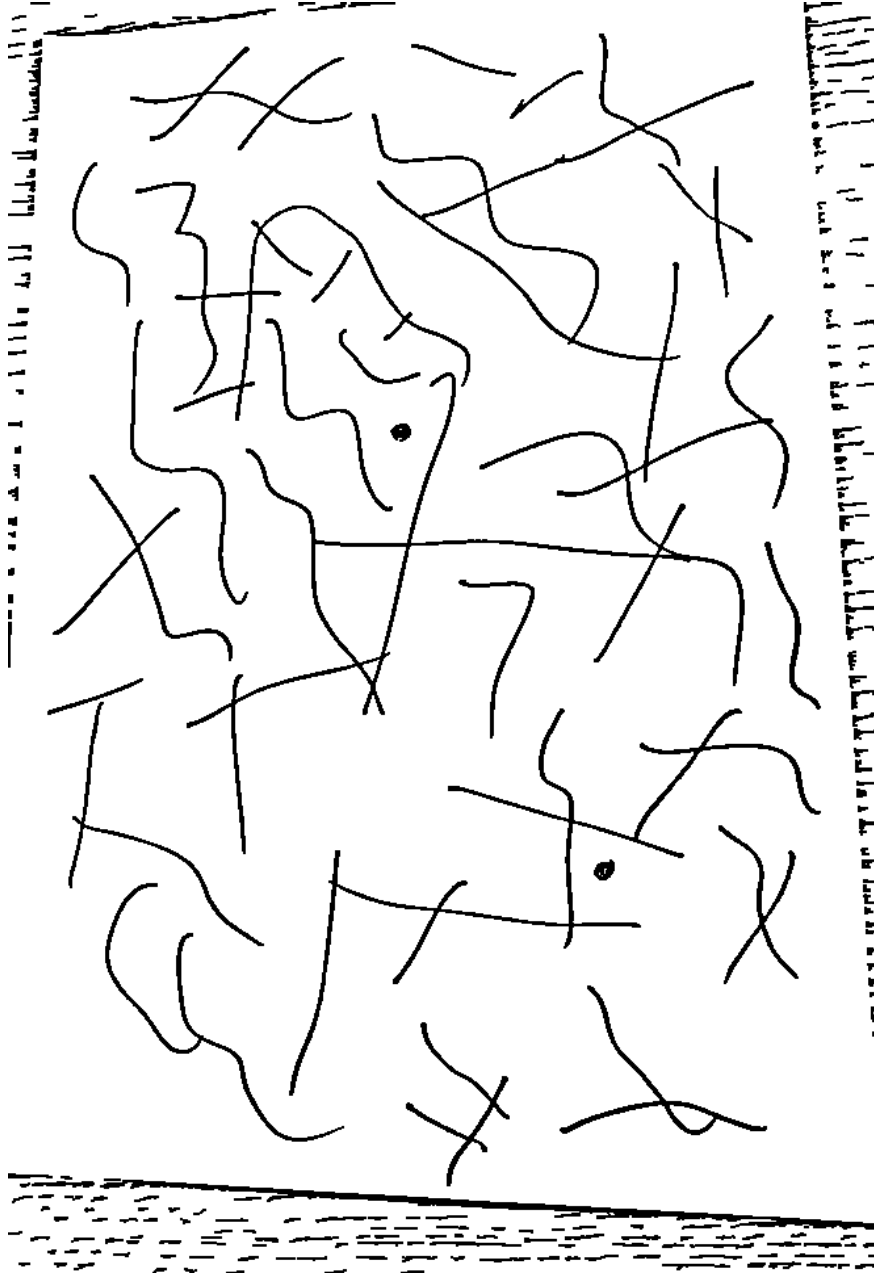


Figure 11: Cleaned binary scan

6 Path Planning

6.1 Mathematical Formulation

Let the rectified binary image have size $W \times H$, and let a pixel be

$$p = (x, y), \quad 0 \leq x < W, \quad 0 \leq y < H.$$

The binary occupancy function is defined as

$$O(x, y) = \begin{cases} 1, & I(x, y) < 128, \text{ wall or obstacle,} \\ 0, & I(x, y) \geq 128, \text{ free space.} \end{cases}$$

The map is downsampled to a grid with maximum side length `grid_max`. With scale

$$s = \frac{\text{grid_max}}{\max(W, H)},$$

the pixel coordinate is mapped to grid coordinate

$$c = (r, c) = (\lfloor sy \rfloor, \lfloor sx \rfloor).$$

The planning graph is

$$G = (V, E), \quad V = \{(r, c) \mid O_g(r, c) = 0\},$$

where O_g is the processed occupancy grid. The objective is

$$P^* = \arg \min_P \sum_{i=1}^{n-1} w(c_i, c_{i+1}), \quad c_1 = c_s, \quad c_n = c_g.$$

6.2 Start and Goal Detection

The start point is marked in red and the goal point is marked in blue. The planner converts the image to HSV space, thresholds red and blue masks, and uses the largest connected component centroid as the marker location.

Listing 4: Red/blue marker detection

```

1 def detect_markers(bgr):
2     hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
3     k = np.ones((5, 5), np.uint8)
4
5     red = cv2.inRange(hsv, (0, 80, 40), (10, 255, 255)) | \
6           cv2.inRange(hsv, (170, 80, 40), (180, 255, 255))
7     red = cv2.morphologyEx(red, cv2.MORPH_OPEN, k)
8
9     blue = cv2.inRange(hsv, (100, 80, 40), (130, 255, 255))
10    blue = cv2.morphologyEx(blue, cv2.MORPH_OPEN, k)
11    return _largest_blob_centroid(red), _largest_blob_centroid(blue)

```

6.3 Occupancy Grid Construction

The wall pixels are processed with marker erasing, morphological closing, conservative downsampling, and obstacle inflation. Obstacle inflation is equivalent to planning in

configuration space:

$$\mathcal{O}_{\text{inflated}} = \mathcal{O} \oplus B_{\rho},$$

where $\oplus$ denotes the Minkowski sum and B_{ρ} is the structuring element with radius ρ .

Listing 5: Occupancy grid construction

```
1 def build_occupancy(binary, markers, close=5, inflate=1, grid_max=400):
2     h0, w0 = binary.shape
3     wall = (binary < 128).astype(np.uint8)
4
5     erase_r = max(12, int(0.02 * max(h0, w0)))
6     for m in markers:
7         if m is not None:
8             cv2.circle(wall, (int(round(m[0])), int(round(m[1]))),
9                           erase_r, 0, -1)
10
11     if close > 0:
12         wall = cv2.morphologyEx(wall, cv2.MORPH_CLOSE,
13                                np.ones((close, close), np.uint8))
14     scale = grid_max / float(max(h0, w0))
15     if scale < 1.0:
16         gh, gw = max(int(h0 * scale), 1), max(int(w0 * scale), 1)
17         wall = (cv2.resize(wall * 255, (gw, gh),
18                           interpolation=cv2.INTER_AREA) > 0).astype(np.uint8)
19     else:
20         scale = 1.0
21     if inflate > 0:
22         wall = cv2.dilate(wall, np.ones((2 * inflate + 1,) * 2, np.uint8))
23     return wall.astype(bool), scale
```

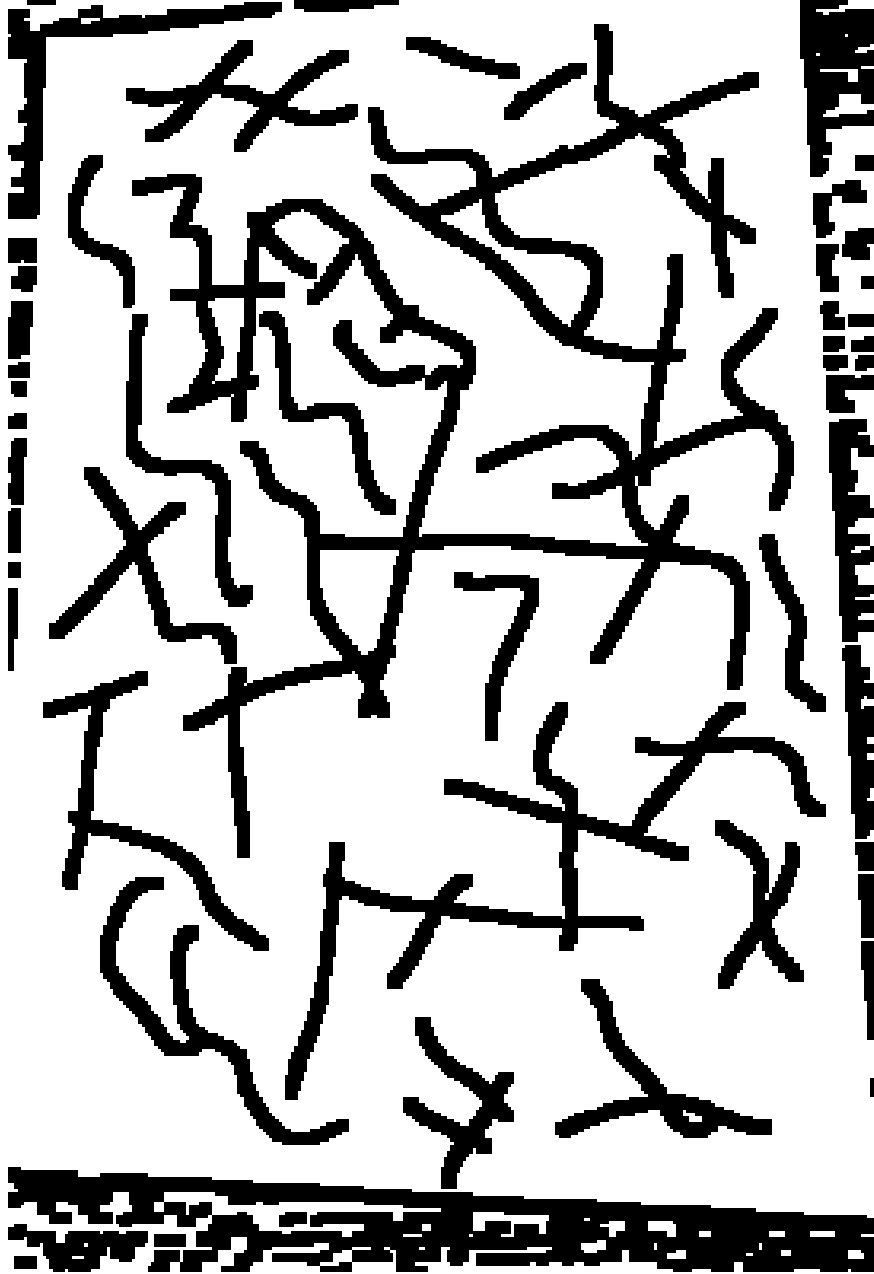


Figure 12: Occupancy grid: white is free space and black is obstacle

6.4 A* Search

The planner uses an 8-connected A* search. The evaluation function is

$$f(n) = g(n) + h(n),$$

where $g(n)$ is the accumulated path cost and $h(n)$ is the Euclidean heuristic:

$$h(n) = \sqrt{(r_n - r_g)^2 + (c_n - c_g)^2}.$$

The Euclidean heuristic is admissible for the 8-connected grid, so A* returns an optimal discrete path. Diagonal moves are additionally checked to prevent cutting through a

corner between two walls.

Listing 6: A* neighbor expansion and diagonal check

```
1 for dr, dc, cost in nbrs:
2     nr, nc = cr + dr, cc + dc
3     if not (0 <= nr < h and 0 <= nc < w) or occ[nr, nc]:
4         continue
5     if dr != 0 and dc != 0 and (occ[cr + dr, cc] and occ[cr, cc + dc]):
6         continue
7     ng = g + cost
8     if ng < gscore.get((nr, nc), float("inf")):
9         gscore[(nr, nc)] = ng
10        came[(nr, nc)] = cur
11        heapq.heappush(open_heap, (ng + heur((nr, nc), goal), ng, (nr, nc)))
```



Figure 13: Planned maze path

7 Kinematics and Trajectory Generation

7.1 Pixel-to-Paper Coordinate Mapping

For a rectified image point (p_x, p_y) with image size (W, H) , the normalized coordinates are

$$u = \frac{p_x}{W}, \quad v = \frac{p_y}{H}.$$

Let the paper size be $L_x = 0.30$ m and $L_y = 0.21$ m, and let the paper center be $(x_c, y_c, z_{\text{paper}})$. The target pen-tip point is

$$\mathbf{x}_d = \begin{bmatrix} x_c + (0.5 - v)L_x \\ y_c + (u - 0.5)L_y \\ z_{\text{paper}} + \Delta z \end{bmatrix},$$

where $\Delta z = 0.001$ m.

Listing 7: Image coordinate to paper coordinate mapping

```

1 PAPER_SX, PAPER_SY, PAPER_TOP = 0.30, 0.21, 0.002
2 PEN_Z = PAPER_TOP + 0.001
3
4 def img_to_world(px, py, wimg, himg, paper_cx, paper_cy):
5     u, v = px / wimg, py / himg
6     return (paper_cx + (0.5 - v) * PAPER_SX,
7             paper_cy + (u - 0.5) * PAPER_SY, PEN_Z)

```

7.2 Arc-Length Resampling

The A* output is a grid polyline. To make the robot trajectory more uniform, the path is resampled by arc length. Given

$$P = \{\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{m-1}\},$$

the cumulative length is

$$l_0 = 0, \quad l_i = \sum_{k=1}^i \|\mathbf{p}_k - \mathbf{p}_{k-1}\|_2.$$

For N waypoints,

$$\hat{l}_j = \frac{j}{N-1} l_{m-1}, \quad j = 0, 1, \dots, N-1.$$

Listing 8: Arc-length resampling

```

1 def resample(pts, n):
2     P = np.asarray(pts, float)
3     d = np.r_[0, np.cumsum(np.linalg.norm(np.diff(P, axis=0), axis=1))]
4     s = np.linspace(0, d[-1], n)
5     return np.c_[np.interp(s, d, P[:, 0]), np.interp(s, d, P[:, 1])]

```

7.3 Forward Kinematics

Let the joint vector be

$$\mathbf{q} = [q_1, q_2, q_3, q_4, q_5]^T.$$

Each joint transform is modeled from the URDF fixed origin and the joint-axis rotation:

$${}^{i-1}T_i(q_i) = \begin{bmatrix} R_i^0 R_z(\sigma_i q_i) & \mathbf{t}_i \\ \mathbf{0}^T & 1 \end{bmatrix}.$$

The end-effector transform is

$${}^0T_5(\mathbf{q}) = \prod_{i=1}^5 {}^{i-1}T_i(q_i).$$

If the pen tip in the link-5 frame is ${}^5\mathbf{p}_{nib}$, then

$${}^0\mathbf{p}_{nib} = {}^0R_5(\mathbf{q}) {}^5\mathbf{p}_{nib} + {}^0\mathbf{t}_5(\mathbf{q}).$$

Listing 9: Forward kinematics

```

1 def fk_M5(q):
2     M = np.eye(4)
3     for (tr, rot, sign), th in zip(_JOINTS, q):
4         M = M @ tr @ rot @ _Rz(sign * th)
5     return M
6
7 def fk_nib_tail(q):
8     M5 = fk_M5(q)
9     o, R = M5[:3, 3], M5[:3, :3]
10    return o + R @ _NIB, o + R @ _TAIL

```

Although a pen-tip Jacobian can be defined as

$$J_p(\mathbf{q}) = \frac{\partial {}^0\mathbf{p}_{nib}}{\partial \mathbf{q}} \in \mathbb{R}^{3 \times 5},$$

the current branch does not explicitly construct this Jacobian. Instead, the forward-kinematics function is directly passed to the SLSQP nonlinear optimizer.

7.4 Inverse Kinematics

The IK solver uses bounded SLSQP optimization. The first stage minimizes position error:

$$\mathbf{q}_{pos} = \arg \min_{\mathbf{q}_{min} \leq \mathbf{q} \leq \mathbf{q}_{max}} \|\mathbf{p}_{nib}(\mathbf{q}) - \mathbf{x}_d\|_2^2.$$

If the target is reachable, the second stage keeps the pen tip fixed and optimizes the pen axis. Let

$$\mathbf{a}(\mathbf{q}) = \frac{\mathbf{p}_{nib}(\mathbf{q}) - \mathbf{p}_{tail}(\mathbf{q})}{\|\mathbf{p}_{nib}(\mathbf{q}) - \mathbf{p}_{tail}(\mathbf{q})\|_2}.$$

The desired axis is downward, $\mathbf{a}_d = [0, 0, -1]^T$, and the orientation objective is

$$\begin{aligned} \mathbf{q}^* = \arg \min_{\mathbf{q}_{min} \leq \mathbf{q} \leq \mathbf{q}_{max}} & (1 + a_z(\mathbf{q})) + \lambda \|\mathbf{q} - \mathbf{q}_{ref}\|_2^2, \\ \text{s.t. } & \mathbf{p}_{nib}(\mathbf{q}) = \mathbf{x}_d. \end{aligned}$$

Listing 10: Inverse kinematics framework

```

1 def ik(target, q0, lim=None):
2     del lim
3     target = np.asarray(target, float)
4     q_ref = np.clip(np.asarray(q0, float), _JOINT_LO, _JOINT_HI)
5
6     def pos_cost(q):
7         err = fk_pos(q) - target
8         return float(err @ err)
9
10    pos_res = minimize(pos_cost, q_ref, method="SLSQP", bounds=_BOUNDS)
11    q_pos = np.clip(pos_res.x if pos_res.x is not None else q_ref,
12                    _JOINT_LO, _JOINT_HI)
13    pos_err = float(np.linalg.norm(fk_pos(q_pos) - target))
14
15    if pos_err > _POS_TOL:
16        a = _pen_axis(q_pos)
17        tilt = np.degrees(np.arccos(np.clip(-a[2], -1, 1)))
18        return q_pos, pos_err, float(tilt)
19
20    def orient_cost(q):
21        a = _pen_axis(q)
22        return float((1.0 + a[2]) + _CONTINUITY_W * np.sum((q - q_ref) ** 2))
23
24    orient_res = minimize(orient_cost, q_pos, method="SLSQP",
25                          bounds=_BOUNDS,
26                          constraints=({"type": "eq",
27                                       "fun": lambda q: fk_pos(q) - target},))
28    q = np.clip(orient_res.x if orient_res.x is not None else q_pos,
29                _JOINT_LO, _JOINT_HI)
30    if np.linalg.norm(fk_pos(q) - target) > max(_POS_TOL, pos_err * 2.0):
31        q = q_pos
32    a = _pen_axis(q)
33    tilt = np.degrees(np.arccos(np.clip(-a[2], -1, 1)))
34    return q, float(np.linalg.norm(fk_pos(q) - target)), float(tilt)

```

The current joint limits are

$$\mathbf{q}_{min} = [-180, -90, -90, -90, -45]^\circ, \quad \mathbf{q}_{max} = [180, 90, 90, 90, 45]^\circ.$$

The fifth joint h is now used as a pen-rotation joint, but the real usable range is still treated as $\pm 45^\circ$.

The current trajectory file contains 70 points. Its metadata is summarized in Table 2.

Table 2: Trajectory metadata

Item	Value
Source image	maze_planner/samples/test_0.jpg
Number of waypoints	70
Paper center distance	paper_cx = 0.22 m
Time interval	dt = 0.1 s
Maximum IK residual	about 0.0494 mm
Maximum pen-axis tilt	about 0.572°

8 Isaac Sim Verification

The simulation module imports the URDF into Isaac Sim and maps the reconstructed maze as a texture on a 30 cm $\times$ 21 cm paper plane. The robot joint positions are set point by point, and a red trail visualizes the drawn path.

Listing 11: Simulation trajectory generation

```

1 pts, warped, binary, start_xy, goal_xy = solve_path(MAZE_IMG, auto=True,
2                                                     debug_dir=_IMG_DIR)
3 path_px = resample(pts, N_WAYPOINTS)
4 targets = [img_to_world(px, py, wimg, himg) for px, py in path_px]
5
6 qtraj, res, tilt = [], [], []
7 q_seed = np.array([0.0, 1.0, 1.0, 0.0, 0.0])
8 for tgt in targets:
9     q_seed, e, ti = ik(np.array(tgt), q_seed)
10    qtraj.append(q_seed.copy())
11    res.append(e)
12    tilt.append(ti)

```

Simulation videos are stored in `arm_sim/video/`, including joint-motion demonstration, circle drawing, and maze drawing.

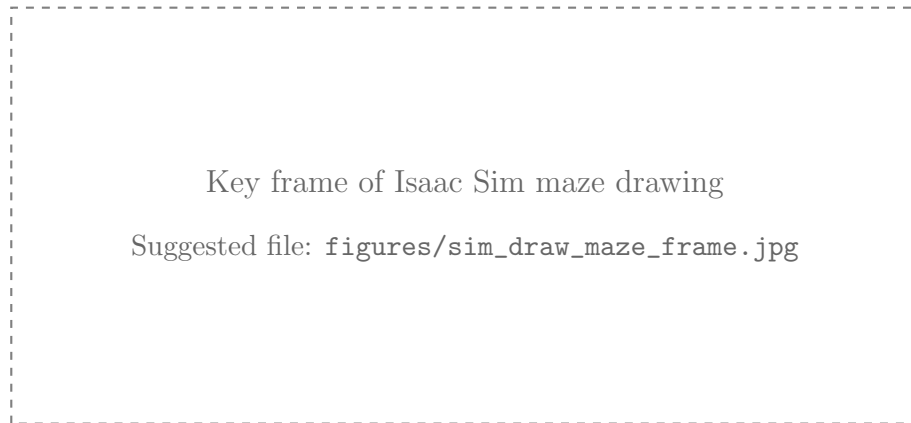


Figure 14: Key frame of Isaac Sim maze drawing, to be added

9 Real Robotic Arm Control

9.1 Communication Architecture

The real system follows an upper/lower-level architecture. The upper-level client performs planning and IK, while the lower-level Windows server communicates with the robot over serial and exposes a TCP JSON API.

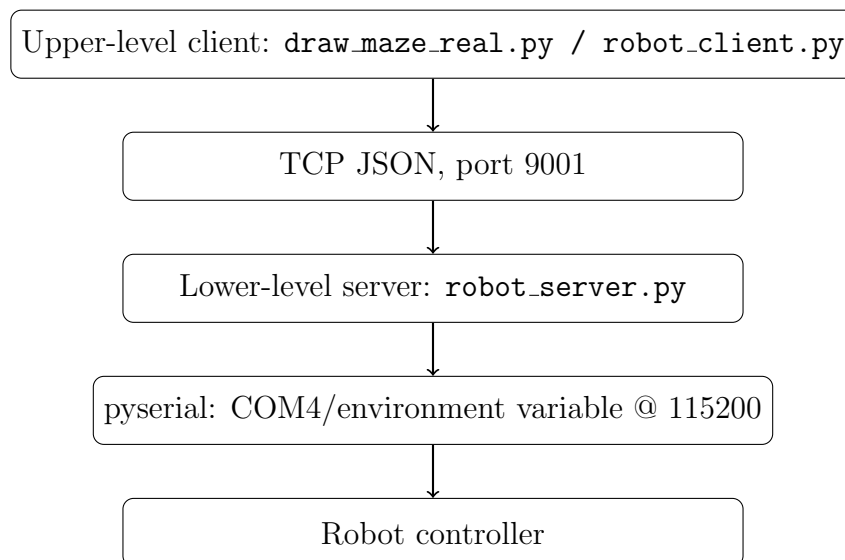


Figure 15: Real-arm communication chain

The current server listens on 0.0.0.0:9001 by default. The serial port defaults to COM4 on Windows and can be overridden with `ROBOT_SERIAL_PORT`. The host and port can also be overridden with `ROBOT_SERVER_HOST` and `ROBOT_SERVER_PORT`.

9.2 TCP Client

Listing 12: Trajectory request in the upper-level client

```
1 class RobotClient:
2     def request(self, payload):
3         msg = json.dumps(payload, separators=(",", ":")) + "\n"
4         with socket.create_connection((self.host, self.port),
5                                     timeout=self.timeout) as sock:
6             sock.sendall(msg.encode("utf-8"))
7             data = sock.recv(4096)
8             return json.loads(data.decode("utf-8"))
9
10    def trajectory(self, points, dt=DEFAULT_DT, traj_id="traj",
11                  spd=DEFAULT_SPD, acc=DEFAULT_ACC):
12        return self.request({
13            "type": "trajectory",
14            "traj_id": traj_id,
15            "dt": dt,
16            "spd": spd,
17            "acc": acc,
18            "points": points,
19        })
```

9.3 Serial Command Generation

The lower-level server converts joint targets into the arm's serial JSON protocol. Joint control uses command type T=122.

Listing 13: Serial joint command generation

```
1 def make_arm_cmd(point, global_spd=None, global_acc=None):
2     cmd = {"T": 122}
3     for joint, (lo, hi) in LIMITS.items():
4         value = point.get(joint, 0.0)
5         cmd[joint] = clamp(value, lo, hi)
6     cmd["spd"] = point.get("spd", global_spd if global_spd is not None else
7                             DEFAULT_SPD)
8     cmd["acc"] = point.get("acc", global_acc if global_acc is not None else
9                             DEFAULT_ACC)
10    return cmd
```


Table 3: Server API

Request	Function
ping	Connectivity test
joint	Send a single joint target
trajectory	Asynchronously execute a full trajectory
status	Query local execution state
state	Query low-level robot state
stop	Stop sending subsequent trajectory points

9.4 Trajectory Execution Strategy

When a trajectory is received, the server starts a worker thread. It first checks whether T=1051 state feedback is available. If feedback works, it uses `poll-until-reached`: send one point, wait until the feedback angles stabilize, then send the next point. If feedback is unavailable, it falls back to `timed(dt)` mode and waits a fixed time after each command.

9.5 Real-Arm Trajectory Preprocessing

The current branch performs two additional preprocessing steps before sending the trajectory. First, backlash compensation can be applied through `--backlash-s-deg` and `--backlash-e-deg`. Shoulder compensation is triggered according to the direction of $|s|$, while elbow compensation is triggered according to the raw direction of e .

Listing 14: Backlash compensation entry point

```

1 def apply_backlash_compensation(points, s_deg, e_deg):
2     n_s = _apply_shoulder_backlash(points, s_deg)
3     n_e = _apply_directional_backlash(points, "e", e_deg)
4     if n_s or n_e:
5         print(f"[backlash] s={s_deg:+.2f}, e={e_deg:+.2f}", flush=True)

```

Second, a pre-turn point is inserted before the actual path. This point uses the first path point's `b/s/e/w` values but keeps `h=0`, reducing sudden end-effector posture changes at the beginning of drawing.

Listing 15: Prepending an `h=0` pre-turn point

```

1 def prepend_w_preturn_point(points):
2     pre = dict(points[0])
3     pre["h"] = 0.0
4     return [pre] + points

```

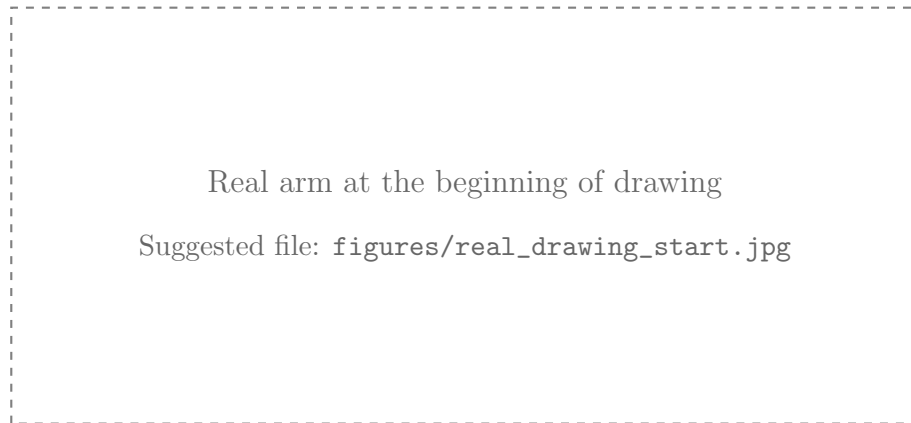


Figure 16: Real arm at the beginning of drawing, to be added

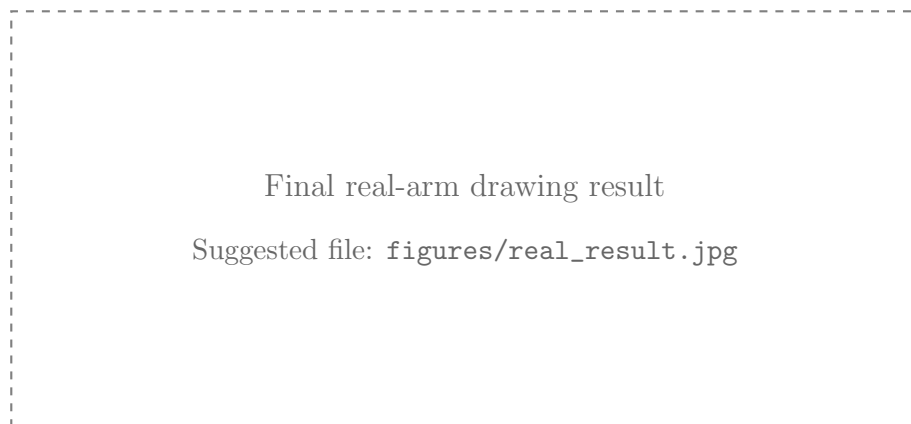


Figure 17: Final real-arm drawing result, to be added

10 Experimental Procedure

10.1 Maze Planning

Listing 16: Maze reconstruction and planning

```
1 conda env create -f maze_planner/environment.yml
2 conda activate maze
3 python maze_planner/maze_planner.py maze_planner/samples/test_0.jpg \
4     -o maze_planner/outputs/image/planned.png --auto
```

10.2 Simulation

Listing 17: Isaac Sim command

```
1 python arm_sim/draw_maze.py
```

10.3 Real-Arm Dry Run

Listing 18: Trajectory generation and dry-run check

```
1 python arm_real_client/draw_maze_real.py --n-waypoints 70
```

The default mode plans, solves IK, checks limits, and saves the trajectory without connecting to hardware. The trajectory is written to `maze_planner/outputs/trajectory/trajectory.json`. Backlash compensation can be added with `--backlash-s-deg` and `--backlash-e-deg`.

10.4 Real-Arm Execution

Listing 19: First hardware test with a few points

```
1 python arm_real_client/draw_maze_real.py --send --from-file --max-points 5
```

Listing 20: Full hardware execution

```
1 python arm_real_client/draw_maze_real.py --send --from-file
```

Listing 21: Software stop

```
1 python arm_real_client/estop.py
```

11 Results and Analysis

11.1 Image Processing Result

The reconstruction pipeline successfully detects the paper region, corrects perspective distortion, and produces a clean binary image suitable for path planning.

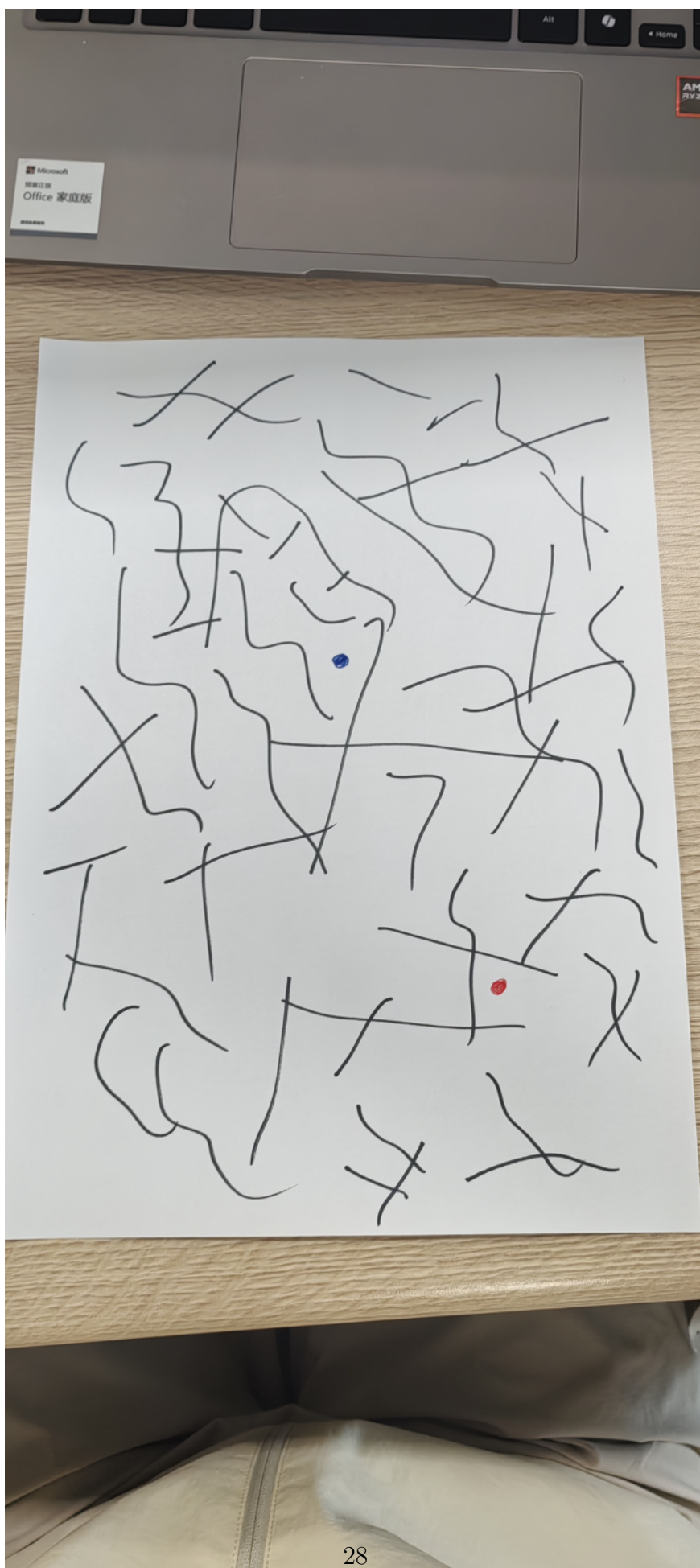


Figure 18: Original input photo



Figure 19: Final planned path

11.2 Path Planning Result

The planned path connects the red start marker to the blue goal marker while avoiding inflated wall regions. The diagonal corner check prevents the algorithm from exploiting unrealistic one-pixel corner gaps.

11.3 Kinematic Result

The current trajectory file reports a maximum IK position residual of approximately 0.0494 mm and a maximum pen-axis tilt of approximately 0.572° . This indicates that the robot model can reach the full maze path with a nearly vertical pen posture under the current paper placement. Before real execution, joint-limit checking is still necessary; if any angle exceeds the real soft limits, the lower-level server clamps the command, which may distort the final drawing.

11.4 Real Hardware Result

The repository does not yet include real hardware photos. After adding them, the result should be evaluated by start-point accuracy, global path offset, corner quality, wall collision, and final goal error.

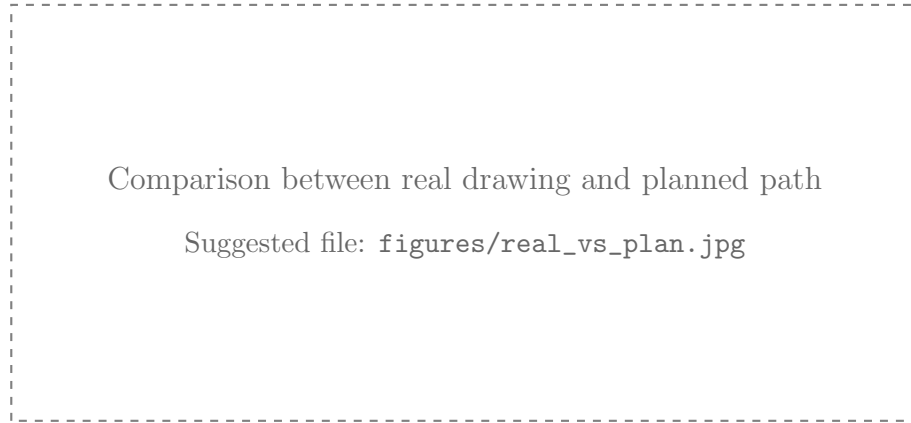


Figure 20: Comparison between real drawing and planned path, to be added

12 Issues and Future Work

First, the paper pose is still manually configured by `paper_cx` and `paper_cy`. Future work can introduce ArUco markers or camera-to-robot calibration.

Second, the real trajectory is executed point by point. This is stable but slow. Continuous trajectory interpolation or velocity planning could make the drawing smoother.

Third, pen pressure is not closed-loop controlled in this branch. A future version could use torque feedback or an external pressure sensor for adaptive z -axis regulation.

Fourth, the pen posture is limited by the robot's joint range, especially the real $\pm 45^\circ$ limit of joint h and the conservative $\pm 90^\circ$ limit of joint w .

Fifth, the vision pipeline still depends on lighting and background conditions. More robust segmentation or fiducial markers could improve reliability.

13 Conclusion

This experiment implements a complete robotic maze drawing system from vision input to real-arm execution. The system reconstructs a paper maze from a camera photo, plans a path using an occupancy-grid A* algorithm, maps the 2-D path to physical paper coordinates, solves bounded inverse kinematics for a 5-DOF robotic arm, verifies the motion in Isaac Sim, and sends the resulting joint trajectory to real hardware through a TCP and serial communication chain.

A Key Files

Table 4: Key project files

File	Role
<code>maze_planner/maze_scanner.py</code>	Paper detection, perspective correction, enhancement, and binarization
<code>maze_planner/maze_planner.py</code>	Marker detection, occupancy-grid construction, and A* planning
<code>arm_sim/arm_kinematics.py</code>	Forward and inverse kinematics for the 5-DOF arm
<code>arm_sim/draw_maze.py</code>	Isaac Sim rendering of maze drawing
<code>arm_real_client/draw_maze_real.py</code>	Converts the maze path to a real-arm joint trajectory
<code>arm_real_client/robot_client.py</code>	TCP client wrapper
<code>arm_real_server/robot_server.py</code>	TCP server and serial command execution
<code>maze_planner/outputs/image/</code>	Intermediate images and planning visualization
<code>maze_planner/outputs/trajectory/</code>	Extra joint trajectory